

Demand Forecasting & Inventory Optimization

An end-to-end, reproducible portfolio project that turns daily sales into demand forecasts and explainable replenishment recommendations. Includes synthetic demo data, validation and cleaning, time-aware model evaluation, inventory policies, Streamlit dashboard, FastAPI endpoint, PostgreSQL SQL, tests, and Docker setup.

*****Demo data:**** The built-in generator produces synthetic retail activity and illustrative supplier/stock assumptions. Model scores, products, financial measures, and recommendations are not observed business results or guaranteed savings. Replace demo inputs with licensed, validated business data before acting on recommendations.*

Business questions

- What did we sell, and how has demand changed?
- What might sell over the selected planning horizon?
- How does a forecast compare with a chronological holdout?
- Which SKUs have stockout or excess-stock risk?
- Under stated lead-time, service-level, and cost assumptions, what quantity might be ordered?

Workflow

Sales + SKU / supplier assumptions

|

v

Validate -> clean -> daily SKU demand panel

|

EDA + lag/calendar features

|

Naive / seasonal-naive / moving average / tree model

|

Chronological backtest: MAE, RMSE, WAPE

|

Forecast -> safety stock -> reorder point -> order quantity

|

Streamlit dashboard / FastAPI / SQL

|

human review before any purchase decision

1. src/demand_inventory/data.py creates deterministic synthetic data or loads CSV files.
2. pipeline.py validates transactions and aggregates returns-adjusted, daily SKU demand.

3. features.py creates lag, rolling, and calendar features using historical data only.
4. forecasting.py compares baselines and a tree regressor on chronological holdouts.
5. inventory.py calculates safety stock, reorder points, EOQ, ABC-XYZ segments, and risk labels.
6. service.py combines the inputs into a daily forecast, evaluation table, and decision table.
7. dashboard/app.py, api/main.py, SQL, and notebooks expose the same workflow.

Quick start (Windows PowerShell)

Python 3.10+ is recommended.

```
py -3.12 -m venv .venv
```

```
.\venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip
```

```
pip install -r requirements.txt
```

```
pip install -e .
```

```
python -m demand_inventory.cli --output-dir data/processed --horizon 30
```

```
streamlit run dashboard/app.py
```

If py is unavailable, create the virtual environment using an installed Python executable. The CLI writes forecasts, evaluation scores, cleaned demand, and inventory decisions under data/processed/.

Run with your own data

Provide two CSV files. The sales CSV must contain order_id,order_date,sku_id,quantity,unit_price,discount,promotion; the products CSV must contain sku_id,product_name,category,unit_cost,current_stock,lead_time_days,minimum_order_quantity,order_cost,annual_holding_cost. Dates should be parseable; returns may be negative quantities. Use the CLI:

```
python -m demand_inventory.cli --sales-csv data/raw/sales.csv --products-csv data/raw/products.csv --horizon 30 --output-dir data/processed
```

Or select **Upload CSV files** in the Streamlit sidebar and upload both tables. Validate the business meaning of on-hand stock, order costs, returns, and lead times before relying on any output.

Run checks:

```
python -m pytest
```

Run the JSON API in a separate terminal:

```
uvicorn api.main:app --reload
```

Open <http://127.0.0.1:8000/docs>.

Application features

The Streamlit dashboard provides demo-data or CSV-upload mode; configurable planning horizon, service level, days-of-supply target, and forecast model; category/SKU/risk filters; historical demand and forecast charts; model evaluation; inventory KPIs; and downloadable recommendation CSV. The FastAPI service

exposes health and synthetic-demo decision endpoints.

Structure

api/ FastAPI application

dashboard/ Streamlit decision-support app

data/ Synthetic inputs and generated outputs (ignored by Git)

docs/ Workflow, methodology, architecture, BI and interview notes

notebooks/ Guided analysis notebooks

publication/ Portfolio, resume and LinkedIn drafts

sql/ PostgreSQL schema and business queries

src/demand_inventory/ Reusable data, feature, model and inventory modules

tests/ Unit and integration tests

Modeling and assumptions

- Rows with invalid dates, unknown SKUs, non-finite numeric values, negative prices, or out-of-range discounts are excluded; duplicate order IDs are deduplicated. Returns are retained as negative transactions and netted against same-day sales; replenishment demand is floored at zero per SKU-day.
- Forecast tests use contiguous, chronological holdout dates, not random splitting. Future tree-model lag features are generated recursively from prior observations/predictions.
- WAPE is $\text{sum}(\text{abs}(\text{actual} - \text{forecast})) / \text{sum}(\text{abs}(\text{actual}))$; it is undefined when total actual demand is zero and represented as blank/None.
- Safety stock uses the documented independent daily-demand/fixed-lead-time approximation: $z \cdot \text{daily-demand standard deviation} \cdot \sqrt{\text{lead time}}$. Real operations should model lead-time demand directly, particularly with variable lead time.
- Reorder point is mean daily demand lead time + safety stock.
- EOQ is $\sqrt{2 \cdot \text{annual demand} \cdot \text{order cost} / \text{annual holding cost per unit}}$. MOQ and pack-size constraints should be added from actual supplier contracts.
- ABC class is based on cumulative revenue proxy; XYZ class uses demand coefficient of variation. Thresholds and caveats are in docs/methodology.md.
- Recommendations are decision support, not purchase orders. Review open orders, expiry, capacity, contract terms, data quality, and local processes.

SQL, Power BI, and Docker

sql/schema.sql provides a PostgreSQL star-schema starting point and sql/business_queries.sql demonstrates reporting patterns. A five-page Power BI design and measures are in docs/powerbi-blueprint.md. A .pbix is not included: a genuine report must be authored and tested in Power BI Desktop.

docker compose up --build

The compose stack starts the dashboard and a local PostgreSQL service. The demo pipeline is file-based and does not require a database connection.

Generated publication package

Install the optional document-generation dependencies and build the PDF manuals/report, Excel data dictionary, 12-slide PowerPoint, and SVG diagrams:

```
pip install -e "[publication]"
```

```
python scripts/build_publication_assets.py
```

Outputs are created in reports/ and may be regenerated from the current source and synthetic holdout. The Power BI blueprint is provided separately; a valid .pbix must be created and checked in Power BI Desktop.

Detailed explanation and publication

Read project explanation, workflow, methodology, data dictionary, and interview guide. Publication drafts are under publication/.

Limits and next steps

This is a reference portfolio, not a production control system. It has no live ERP ingestion, authenticated access, purchase-order execution, promotion-plan feed, calibrated prediction intervals, or alerting. A production rollout requires data contracts, SKU-level backtesting, monitoring, approval controls, access management, and operational sign-off.

License

MIT. See LICENSE.